Developer Guide

Customization Update 2026 R1

Acumatica
The Cloud ERP

Contents

Copyright 3

Validating Customization Projects 4

Validation of Customization Projects: General Information 4

Validation of Customization Projects: Compatibility Validation 4

Validation of Customization Projects: Compatibility Validation Errors 6

Validation of Customization Projects: Using AI Tools 7

Validation of Customization Projects: Ignore List for the Compatibility Validation Errors 8

Validation of Customization Projects: ISV Solutions 8

Validation of Customization Projects: Certification Status of Projects 10

Updating Customization Projects to Acumatica ERP 2026 R1 12

Customization Update to 2026 R1: Multiple Implementations of a Generic Graph Extension in One Graph 12

Customization Update to 2026 R1: Support for Dependency Injection in the Credit Card Processing Framework 14

Updating Customization Projects for the Modern UI 19

Updating a Customization Project for the Modern UI: General Information 19

Updating a Customization Project for the Modern UI: Customization Project Converter 21

Updating a Customization Project for the Modern UI: To Convert a Customized Form 24

Updating a Customization Project for the Modern UI: Form Converter 28

Updating a Customization Project for the Modern UI: To Convert a Custom Form 29

Copyright

© 2026 Acumatica, Inc.

ALL RIGHTS RESERVED.

No part of this document may be reproduced, copied, or transmitted without the express prior consent of Acumatica, Inc.

3075 112th Avenue NE, Suite 200, Bellevue, WA 98004, USA

Restricted Rights

The product is provided with restricted rights. Use, duplication, or disclosure by the United States Government is subject to restrictions as set forth in the applicable License and Services Agreement and in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.227-7013 or subparagraphs (c)(1) and (c)(2) of the Commercial Computer Software-Restricted Rights at 48 CFR 52.227-19, as applicable.

Disclaimer

Acumatica, Inc. makes no representations or warranties with respect to the contents or use of this document, and specifically disclaims any express or implied warranties of merchantability or fitness for any particular purpose. Further, Acumatica, Inc. reserves the right to revise this document and make changes in its content at any time, without obligation to notify any person or entity of such revisions or changes.

Trademarks

Acumatica is a registered trademark of Acumatica, Inc. HubSpot is a registered trademark of HubSpot, Inc. Microsoft Exchange and Microsoft Exchange Server are registered trademarks of Microsoft Corporation. All other product names and services herein are trademarks or service marks of their respective companies.

Software Version: 2026 R1

Last Updated: 03/15/2026

Validating Customization Projects

A customization project must meet various technical requirements to integrate with and operate within the Acumatica ERP application. Also, the Acumatica ERP code is continuously updated to add or enhance functionality —so a customization project's code can become inconsistent with the code of an upgraded instance of Acumatica ERP. To make sure your customization project stays compatible, you can validate customization projects.

Validation of Customization Projects: General Information

By validating customization projects, you can ensure that they can be published in the current Acumatica ERP instance, meet specific requirements, and have passed technical validation in Acumatica.

Learning Objectives

In this chapter, you'll learn how to do the following:

- Validate customization projects for compatibility with the new version before an upgrade
- Validate customization projects of independent software vendor (ISV) solutions before submitting them for technical validation
- Check whether customization projects have passed technical validation by Acumatica

Applicable Scenarios

You validate a customization project in the following cases:

- You have a customization project that works properly for a version of Acumatica ERP, and you need to upgrade an application instance to a newer version. Before the upgrade, you need to validate the customization project with the newer version. For details on this scenario, see *Validation of Customization Projects: Compatibility Validation*.
- As a developer for an ISV partner of Acumatica ERP, you have developed a customization project, which you need to submit to Acumatica for technical validation. To successfully pass this validation, you first need to validate the customization project by using Acumatica ERP validation tools. To learn about the available validation techniques, see *Validation of Customization Projects: ISV Solutions*.
- You're a system administrator who needs to check whether the customization projects you're going to publish on an Acumatica ERP instance have passed Acumatica's technical validation. For more information, see *Validation of Customization Projects: Certification Status of Projects*.

Validation of Customization Projects: Compatibility Validation

To prevent issues between a customization project and a new version of Acumatica ERP, you need to validate the customization project before upgrading a customized instance of Acumatica ERP. During publication, the Acumatica Customization Platform validates the compatibility of the project's code with the application code of the instance.

By default, the validation that detects breaking changes is turned on in an instance of Acumatica ERP. If you need to **temporarily** omit this validation, you can turn off the `CheckCustomizationCompatibility` key by including the following string in the `<appSettings>` section of the `web.config` file located in the website folder.

```
<add key="CheckCustomizationCompatibility" value="False" />
```

We don't recommend turning off the validation permanently.

Before the Upgrade

In a development environment, test your customizations before a production upgrade to Acumatica ERP 2026 R1:

1. Deploy a new instance of Acumatica ERP 2026 R1. (See *Deploying Acumatica ERP Instances* for details.)
2. On the *Customization Projects* (SM204505) form, import all customization projects that you need to upgrade for Acumatica ERP 2026 R1.
3. Start the publication of these customization projects. The system first validates the projects for compatibility with the current version.

If validation errors occur:

- Learn what each error message means. (See *Validation of Customization Projects: Compatibility Validation Errors*.)
- Check the developer release notes for descriptions of specific issues. You can find the release notes, along with the needed Acumatica ERP build, on the *Acumatica Community* website.
- Replace updated or deprecated objects with the recommended alternatives. You can use AI tools to simplify update. (See *Validation of Customization Projects: Using AI Tools*.)
- Redesign solutions if no alternatives are available for removed objects.

During Validation

This validation process executes the following checks in the code of a customization project to detect the breaking changes in the code of Acumatica ERP:

- In graph extensions:
  - Checking the signature for each method that is overridden by using the `PXOverride` attribute
  - Checking that each base graph exists
- In data access class extensions when a field attribute is overridden:
  - Checking that the field exists
  - Checking that the field type hasn't changed
- In binary DLL files: Checking all the referenced methods, properties, fields, return types, and signatures

If the validation has completed successfully, you can upgrade an instance of Acumatica ERP. For details, see *Upgrading of Acumatica ERP: General Information*.

During the Upgrade

In the production environment, do the following during the upgrade to Acumatica ERP 2026 R1:

1. Unpublish all customization projects. (See *Unpublishing Customization Projects* for details.)
2. Upgrade the Acumatica ERP instance. (Learn more in *Upgrading of Acumatica ERP: General Information*.)

3. Replace the old customization projects with those validated for Acumatica ERP 2026 R1. (See *To Replace the Content of a Project from a Package* for more information.)
4. Publish the validated customization projects. (See *Publishing Customization Projects* for details.)

Validation of Customization Projects: Compatibility Validation Errors

This topic explains the error messages that may occur during the validation of the customization code's compatibility with the code of Acumatica ERP.

Failed to resolve method reference

This error occurs if a custom DLL has a reference to a method that either no longer exists in Acumatica ERP or has a different signature.

Example: The following error message indicates that the `AM.Objects.dll` file contains a reference to the `System.Void PX.Data.PXLineNbrAttribute::.ctor(System.Type)` method, which hasn't been declared or has another signature in the `PX.Data` assembly.

```
AM.Objects.dll
Failed to resolve method reference:
System.Void PX.Data.PXLineNbrAttribute::.ctor(System.Type)
declared in
PX.Data, Version=1.0.0.0, Culture=neutral, PublicKeyToken=3b136cac2f602b8e
```

Because the customization code was written for a previous version of Acumatica ERP, this error occurs because the method overridden in the customization code has since been removed or its signature has been changed. To confirm that this change is implemented in the new version of Acumatica ERP, check the developer release notes for this version. You can find the release notes, along with the needed Acumatica ERP build, on the *Acumatica Community* website.

To fix the error: In the code of the specified extension library, refer to an appropriate method declared in the current assembly.

Failed to resolve type reference

This error occurs if a custom DLL refers to a type whose declaration no longer exists in Acumatica ERP.

Example: The following error message means that the `FETempFix.dll` file contains a reference to the `PX.Data.PXGraphWithActionsBase`2` type, which is not declared in the `PX.Data` assembly.

```
FETempFix.dll
Failed to resolve type reference:
PX.Data.PXGraphWithActionsBase`2
declared in
PX.Data, Version=1.0.0.0, Culture=neutral, PublicKeyToken=3b136cac2f602b8e
```

To fix the error: In the extension library, refer to an appropriate type declared in the current assembly.

Could not resolve

This error occurs if the validation process finds a reference to a custom DLL that it can't locate.

Example: The following error message tells you that the `FullRegen.dll` file contains the `ADODB` reference, which cannot be resolved.

```
FullRegen.dll
Could not resolve:
ADODB, Version=7.0.3300.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a
```

To fix the error: In the extension library, refer to a DLL file located in the `Bin` folder of the website.

Declaring Type missing

This error occurs if the customization code in the `App_RuntimeCode` folder contains a reference to a type whose declaration doesn't exist in Acumatica ERP.

Example: The following error message indicates that the `PX.Objects.EP.CRBaseActivityMaint`1<PX.Objects.CR.CRTaskMaint>` class refers to the undeclared `BaseBAccount` type.

```
Declaring Type missing:
BaseBAccount
from
PX.Objects.EP.CRBaseActivityMaint`1<PX.Objects.CR.CRTaskMaint>
```

To fix the error: In the corresponding *Code* or *DAC* item of the customization project, refer to an appropriate type.

Referenced Field missing

This error occurs if the customization code in the `App_RuntimeCode` folder contains a field that's undeclared in the data access class (DAC).

Example: The following error message tells you that the `PX.Objects.CS.CSAnswers` DAC doesn't contain the `_EntityID` field.

```
Referenced Field missing:
_EntityID
from
PX.Objects.CS.CSAnswers
```

To fix the error: In the corresponding *Code* or *DAC* item of the customization project, refer to an existing field.

Validation of Customization Projects: Using AI Tools

Suppose that you have a customization project that works properly for Acumatica ERP 2025 R2, and you need to upgrade an application instance to 2026 R1. Before the upgrade, you validate the customization project with the newer version, as described in *Validation of Customization Projects: Compatibility Validation*. If validation errors occur, you need to fix these errors.

You can use an AI tool, such as GitHub Copilot, to speed up and simplify fixing of errors.

Using an AI Tool in Visual Studio

You can use the following general steps to update your customization project by using an AI tool:

1. Review the developer release notes to understand the changes in the new version.
2. Open your customization project's extension library in Visual Studio.
3. Add the **Markdown** version of the developer release notes from the *Acumatica Community* website to the project of the extension library.

4. Open a file with the Acumatica ERP-specific code, such as a graph or DAC extension.
5. Launch a chat with your AI tool.
6. Write a prompt, such as *Update this class to Acumatica ERP 2026 R1, use #<project name>/ AcumaticaERP_2026R1_ReleaseNotes_for_Developers.md*.

   > Instead of referencing the file in the prompt, you can copy and paste the contents of the file to the chat.

7. Wait for the AI agent to process the context.
8. Carefully review the provided recommendations and apply valid ones. Double-check them with the developer release notes if you aren't sure about the changes.
9. Use the same context for other files in the project of your extension library.

Validation of Customization Projects: Ignore List for the Compatibility Validation Errors

If the validation of the compatibility of the published customization with the new version fails, the platform cancels the upgrade process and shows an error message. However, if you're sure that a generated error isn't really an error, you can force the validation process to ignore it so that you can continue upgrading of the customized instance of Acumatica ERP. For example, you may need to ignore an error in a third-party DLL file.

To force the validation process to ignore an error, you perform the following actions:

1. In the `App_Data` folder of the website, create a `.txt` file whose name has the *CstValidationIgnore* prefix, such as `CstValidationIgnore.txt` or `CstValidationIgnore_ProjectName.txt`.

   > You can create multiple `CstValidationIgnore*.txt` files with ignore lists for an instance of Acumatica ERP.

2. In the file, paste the error message that you want to ignore.

   > You can add multiple error messages to each file with ignore lists.

3. Save the file.
4. Determine which customization project has the code that provokes the error message.
5. Add the file with the ignore list to the project (see *To Add a Custom File to a Project* for details).

   > If a file with a *CstValidationIgnore* prefix is placed in the `App_Data` folder, the Acumatica Customization Platform will ignore the specified errors during validation.

As a result, if you publish the customization project or upgrade an Acumatica ERP instance where the project is published, the validation process ignores the error that is specified in the file.

Validation of Customization Projects: ISV Solutions

The validation of a customization project is especially important when the project is a part of an independent software vendor (ISV) solution.

> To learn the technical requirements for an ISV customization project, open the Acumatica Partner Portal and click **ISV Program > ISV Solution Assessment**.

To prepare your solution for technical validation, you can use the tools and validation techniques described below.

Validation of Customization Projects

You can validate any number of customization projects to be sure each project meets the following criteria:

- The database schema changes included in the customization project are valid.
- The customization project does not replace or modify any files distributed as part of Acumatica ERP.
- The data types of custom database columns correspond to the data access class (DAC) fields bound to them.

You can perform this type of validation on the *Customization Projects* (SM204505) form. On the More menu (under **Validations**), click **Validate Highlighted Project** or **Validate Multiple Projects**.

Validation of Data Consistency in Published Customization Projects

The validation of data consistency in published customization projects checks whether the data types of DAC fields match the field states generated by field attributes at runtime. To perform this validation, you use the commands on the More menu (under **Validations**) of the *Customization Projects* (SM204505) form as follows:

- To verify that the data types of DAC fields match the field states generated at runtime, click **DAC Field Types (Runtime Validation)**.
- To check for `PXAttributeFamilyAttribute` violations on DAC fields, click **DAC Attributes (Runtime Validation)**.
- To verify that `PXDimensionAttribute` attributes properly handle foreign key segments, click **Lookup Definitions (Runtime Validation)**.

Validation of a Project Prefix

In the Customization Project Editor, you can specify a prefix that should be added to all objects of the customization project. If a prefix is used, all project item names must start with it.

When validation of the customization project is performed, the platform checks all objects to be sure they have the prefix.

To add a prefix for a customization project and validate the project's items for the added prefix, do the following:

1. On the main menu of the Customization Project Editor, select **File > Validate Project Prefix**.

   The **Customization Project Prefix** dialog box opens.
2. In the **Project Prefix** box of the dialog box, type the prefix for the customization project. The prefix can be two to four case-insensitive Latin letters (*A* through *Z*).
3. Click **Save**.
4. Click **Validate Project Items**. In the **Validation Result** area, the result of the validation appears.

Format of the Validation Results

If any errors or warnings have been detected during project validation, the validation results have the following format:

- <CustomizationProjectName>
  - <CustomizationRuleName>
    - <PathToTheDLL> | <ProjectItemType> | <NameOfTheProjectItem>

List of rules broken down by the specified item

> The `<PathToTheDLL>` field is optional and appears only if a `dll` file included in the project is analyzed.

Analysis of Assemblies Included in the Customization Project

You can analyze assemblies included in a customization project to verify that the project doesn't replace or modify files distributed as part of Acumatica ERP.

To run the analysis, on the main menu of the Customization Project Editor, click **Extension Library > Analyze Referenced Assemblies**. A CSV file is downloaded with all references made in the project's custom files.

Generation of a Workbook with Project Items

When you submit your solution for technical validation in Acumatica, you need to provide an Excel workbook with project items. You can generate this workbook in the Customization Project Editor by clicking **File > Export Project Items Workbook**.

The generated workbook contains a list of solution objects and integration scenarios grouped into the following types:

- Site map nodes
- Mobile site map nodes
- BLCs and BLC extensions
- DACs and DAC extensions
- Push notifications
- Import or export scenarios

Validation of Customization Projects: Certification Status of Projects

If a customization project doesn't pass technical validation, it may negatively affect the performance of Acumatica ERP. To prevent this performance degradation, the system verifies the certification status of the customization projects that you create or upload.

> This functionality is available for only licensed Acumatica ERP instances and is turned off by default. To use it, contact your Acumatica support provider.

Validating the Certification Status of Customization Projects

To validate the certification status of customization projects, you perform the following steps:

1. Open the *Customization Projects* (SM204505) form.
2. On the More menu (under **Validations**), click **Validate Certification Status**.

   The system performs the check and displays the result in the **Certification Status** column for each project.

Each customization project has one of these certification statuses in the Acumatica ERP instance:

- *Certified*: The project has been imported and is available in the database with the certified customization projects.

- *Not Certified*: The project has been imported but isn't available in the database with the certified customization projects.
- *Not Verified*: The project has been:
  - Uploaded before the upgrade to 2026 R1
  - Created manually
  - Certified—but then a customizer modified it
- *Not Certified for Current Version*: The project has been imported and is available in the database with the certified customization projects. However, it was certified for a different major version than the major version of the current Acumatica ERP instance.

If a project isn't certified or its certification status can't be verified, the system displays a warning icon to the right of its name.

If you modify a certified customization project, its certification status will change to *Not Verified*.

We recommend that you avoid publishing a customization project with the *Not Certified*, *Not Verified*, or *Not Certified For Current Version* certification status. If you decide to publish one, the system will display a warning during publication.

Updating Customization Projects to Acumatica ERP 2026 R1

In this chapter, you can find details on the major changes in Acumatica ERP 2026 R1 that affect customization projects. The full list of changes is available in the developer release notes. You can find the release notes, along with the needed Acumatica ERP build, on the *Acumatica Community* website.

Customization Update to 2026 R1: Multiple Implementations of a Generic Graph Extension in One Graph

In Acumatica ERP, you can reuse business logic by using generic graph extensions. Acumatica ERP 2026 R1 lets you go a step further: Now you can implement the same generic graph extension in a graph multiple times.

Suppose that in the custom `RSSVOrderEntry` graph, you need multiple implementations of the `SalesPriceGraph` generic graph extension, each with a different mapping of the `Detail` mapped cache extension. In one implementation class, you need to map `Detail` to the `RSSVOrderActual` data access class (DAC). In another implementation class, you map it to the `RSSVOrderDetail` DAC. The sections below show how you can do this.

Changes to the Generic Graph Extensions

A definition of reusable business logic with generic graph extensions typically includes the following parts:

- The generic graph extension itself, which is a `PXGraphExtension` class
- One or more mapped cache extensions, which are `PXMappedExtension` classes
- One or more mapping classes, which are `IBqlMapping` implementations

To support multiple implementations of generic graph extensions in one graph, Acumatica ERP 2026 R1 introduces the following changes:

- Generic graph extensions now can include multiple base DACs as their type parameters.
- Mapped cache extensions and mapping classes are nested within the generic graph extension.

The following code shows an example of a generic graph extension. Notice that the code includes simplified versions of the generic graph extension, which have fewer type parameters.

```
// A generic graph extension
public abstract partial class SalesPriceGraph<
    TGraph, TDocument, TDetailOpt, TPriceClassSourceOpt> :
    PXGraphExtension<TGraph>
        where TGraph : PXGraph
        where TDocument : class, IBqlTable, new()
        where TDetailOpt : class, IBqlTable
        where TPriceClassSourceOpt : class, IBqlTable
{
    // Mapped cache extensions
    public class Document : PXMappedCacheExtension
    public class Detail : PXMappedCacheExtension
    public class PriceClassSource : PXMappedCacheExtension

    // Mapping classes
    protected class DocumentMapping : IBqlMapping
    protected class DetailMapping : IBqlMapping
```

```
    protected class PriceClassSourceMapping : IBqlMapping
}

// Simplified versions of the generic graph extension
public abstract class SalesPriceGraph<TGraph, TPrimary> :
    SalesPriceGraph<TGraph, TPrimary, IBqlTable, IBqlTable>
    where TGraph : PXGraph
    where TPrimary : class, IBqlTable, new()
{ }

public abstract class SalesPriceGraph<TGraph, TDocument, TDetail> :
    SalesPriceGraph<TGraph, TDocument, TDetail, IBqlTable>
    where TGraph : PXGraph
    where TDocument : class, IBqlTable, new()
    where TDetail : class, IBqlTable, new()
{ }
```

Multiple Implementations in One Graph

To connect the generic graph extension to a base graph, in the base graph, you define an implementation class that inherits the generic graph extension. Starting in Version 2026 R1, you can define multiple implementation classes in one graph as follows (shown in the code below):

- In the `SalesPrice` implementation class, you map the `Detail` mapped cache extension to the `RSSVOrderActual` DAC.
- In the `SalesPriceActual` implementation class, you map the `Detail` mapped cache extension to the `RSSVOrderDetail` DAC.

```
public abstract class SalesPriceWithoutDetail<TDetail> :
    SalesPriceGraph<RSSVOrderEntry, RSSVOrder, TDetail>
    where TDetail : class, IBqlTable, new()
{
    #region Initialization

    protected override DocumentMapping GetDocumentMapping()
    {
        return new DocumentMapping(typeof(RSSVOrder))
        {
            BAccountID = typeof(RSSVOrder.customerID),
            DocumentDate = typeof(RSSVOrder.docDate),
        };
    }

    protected override PriceClassSourceMapping GetPriceClassSourceMapping()
    {
        return new PriceClassSourceMapping(typeof(Location))
        {
            PriceClassID = typeof(Location.cPriceClassID)
        };
    }
    #endregion

    protected override void RecalculateMarkup(PXCache sender, Detail row) { }
}

public class SalesPrice : SalesPriceWithoutDetail<RSSVOrderDetail>
{
```

```
    public static bool IsActive() => true;

    protected override DetailMapping GetDetailMapping()
    {
        return new DetailMapping(typeof(RSSVOrderDetail))
        {
            LocationID = typeof(RSSVOrderDetail.customerLocationID),
            CuryLineAmount = typeof(RSSVOrderDetail.curyExtPrice),
            Descr = typeof(RSSVOrderDetail.tranDesc),
            Quantity = typeof(RSSVOrderDetail.estimatedQty)
        };
    }
}

public class SalesPriceActual : SalesPriceWithoutDetail<RSSVOrderActual>
{
    public static bool IsActive() => true;

    protected override DetailMapping GetDetailMapping()
    {
        return new DetailMapping(typeof(RSSVOrderActual))
        {
            LocationID = typeof(RSSVOrderActual.customerLocationID),
            CuryLineAmount = typeof(RSSVOrderActual.curyExtPrice),
            Descr = typeof(RSSVOrderActual.description),
            Quantity = typeof(RSSVOrderActual.actualQty),
            CuryUnitCost = typeof(RSSVOrderActual.curyStdCost),
            CuryUnitPrice = typeof(RSSVOrderActual.curyUnitPrice),
        };
    }
}
```

Related Links

- *Generic Graph Extensions: General Information*

Customization Update to 2026 R1: Support for Dependency Injection in the Credit Card Processing Framework

In Acumatica ERP 2026 R1, the credit card processing framework has been refactored to support **constructor-based dependency injection (DI)** for payment plug-ins. This refactoring gives you modern tools for creating more efficient, modular, and testable integrations.

The Big Picture

Previously, credit card processing plug-ins relied on the `ServiceLocator` pattern to resolve dependencies. These approaches limited testability and made plug-ins harder to maintain. The following code shows an example of such an approach.

```
protected async Task<HttpResponseMessage> ExecuteAsync(HttpRequestMessage request)
{
 using (var client = ServiceLocator.Current
                      .GetInstance<IHttpClientFactory>().CreateClient())
 {
  var response = await client.SendAsync(request);
```

```
  return response;
 }
}
```

In Version 2026 R1, the new framework replaces this pattern with **constructor-based dependency injection**. As a result, you can implement payment integrations that are **scalable and DI-friendly**.

Dependency Injection for Plug-In Creation

Plug-in dependencies are now injected directly through constructors. Instead of resolving plug-ins through `ServiceLocator`, the framework resolves them through a factory managed by the DI container.

To create a plug-in object, the framework provides a plug-in factory called `CCProcessingPluginFactory` in the `PX.Objects.CC.Factories` namespace. In its constructor, a dictionary of all registered plug-in types is injected, and an instance of a plug-in is received by the plug-in's fully qualified type name. You can use this factory when you need to create plug-in instances explicitly.

The following code snippet shows the implementation of the plug-in factory.

```
public class CCProcessingPluginFactory: ICCProcessingPluginFactory
{
  private readonly IIndex<string, ICCProcessingPlugin> _pluginIndex;

  public CCProcessingPluginFactory(IIndex<string,
                                   ICCProcessingPlugin> pluginIndex)
  {
    _pluginIndex =
      pluginIndex ?? throw new ArgumentNullException(nameof(pluginIndex));
  }

  public ICCProcessingPlugin CreatePluginForProcessingCenter
                                   (CCProcessingCenter processingCenter)
  {
    try
    {
      Type pluginType =
           CCPluginTypeHelper.GetPluginTypeWithCheck(processingCenter);
      return CreatePluginForPluginType(pluginType.FullName);
    }
    catch (PXException)
    {
      throw;
    }
    catch (Exception ex)
    {
      throw new PXException(ex,
                AR.Messages.ERR_ProcessingCenterTypeInstanceCreationFailed,
                            processingCenter.ProcessingTypeName,
                            processingCenter.ProcessingCenterID);
    }
  }

  public ICCProcessingPlugin CreatePluginForPluginType(
                             string pluginTypeFullName)
  {
    if (string.IsNullOrEmpty(pluginTypeFullName))
      {
        throw new ArgumentNullException(nameof(pluginTypeFullName));
```

```
      }
    if (!_pluginIndex.TryGetValue(pluginTypeFullName,
                                  out ICCProcessingPlugin plugin))
      {
        throw new PXException(
              AR.Messages.ERR_ProcessingCenterTypeInstanceCreationFailed,
               pluginTypeFullName, "Unknown");
      }
      return plugin;
  }
}
```

Plug-In Registration

The system automatically registers all plug-ins as services in the DI container. So if you develop a new plug-in, no additional registration steps are required.

To register plug-in types, the system checks available assemblies (by using the `PXProviderTypeSelectorAttribute.GetTypesImplementing` method) for implementations of the `CCProcessingBase.Interfaces.V2.ICCProcessingPlugin` interface. The system then registers every type as a named service. The following code shows an example.

```
using System;
using Autofac;
using PX.CCProcessingBase.Interfaces.V2;
using PX.CCProcessingBase.Logging.V2;
using PX.Objects.CC.Factories;
using PX.Objects.Common;

namespace PX.Objects.AR.CCPaymentProcessing
{
  internal class ServiceRegistration : Module
  {
   protected override void Load(ContainerBuilder builder)
   {
     builder.RegisterType<CCProcessingPluginFactory>()
      .As<ICCProcessingPluginFactory>().SingleInstance();

     foreach (var type in PXProviderTypeSelectorAttribute
              .GetTypesImplementing(typeof(ICCProcessingPlugin)))
     {
        builder.RegisterType(type)
         .Named<ICCProcessingPlugin>(type.FullName);
     }
      builder.RegisterDecorator<
        LoggingProcessingPlugin, ICCProcessingPlugin>();
   }
  }
}
```

Note that this approach works for both built-in plug-ins and third-party plug-ins delivered as part of a customization project.

Upgrade Notes

As a result of the credit card processing framework being refactored to support dependency injection, changes have been made to some data access classes (DACs) and methods.

The `PX.Objects.CA.PXProviderTypeSelectorAttribute.ProviderRec` DAC has been replaced with `PX.Objects.Common.PXProviderTypeSelectorAttribute.ProviderRec`.

The following methods have been replaced:

- In the `PX.Objects.AR.CCPaymentProcessing.CardsSynchronization` namespace:
  - `CCSynchronizeCardManager.ctor(PXGraph, String, CreditCardReceiverFactory)` with `CCSynchronizeCardManager.ctor(PXGraph, String, CreditCardReceiverFactory, ICCProcessingPluginFactory)`
- In the `PX.Objects.AR.CCPaymentProcessing` namespace:
  - `CCCustomerInformationManager.GetCreatePaymentProfileForm(PXGraph, ICCPaymentProfileAdapter)` with `CCCustomerInformationManager.GetCreatePaymentProfileForm(PXGraph, ICCPaymentProfileAdapter, ICCProcessingPluginFactory)`
  - `CCCustomerInformationManager.GetNewPaymentProfiles(PXGraph, ICCPaymentProfileAdapter, ICCPaymentProfileDetailAdapter)` with `CCCustomerInformationManager.GetNewPaymentProfiles(PXGraph, ICCPaymentProfileAdapter, ICCPaymentProfileDetailAdapter, ICCProcessingPluginFactory)`
  - `CCCustomerInformationManager.GetManagePaymentProfileForm(PXGraph, ICCPaymentProfile)` with `CCCustomerInformationManager.GetManagePaymentProfileForm(PXGraph, ICCPaymentProfile, ICCProcessingPluginFactory)`
  - `CCCustomerInformationManager.ProcessProfileFormResponse(PXGraph, ICCPaymentProfileAdapter, ICCPaymentProfileDetailAdapter, String)` with `CCCustomerInformationManager.ProcessProfileFormResponse(PXGraph, ICCPaymentProfileAdapter, ICCPaymentProfileDetailAdapter, ICCProcessingPluginFactory, String)`
  - `CCCustomerInformationManager.GetOrCreatePaymentProfile(PXGraph, ICCPaymentProfileAdapter, ICCPaymentProfileDetailAdapter)` with `CCCustomerInformationManager.GetOrCreatePaymentProfile(PXGraph, ICCPaymentProfileAdapter, ICCPaymentProfileDetailAdapter, ICCProcessingPluginFactory)`
  - `CCCustomerInformationManager.GetPaymentProfile(PXGraph, ICCPaymentProfileAdapter, ICCPaymentProfileDetailAdapter)` with `CCCustomerInformationManager.GetPaymentProfile(PXGraph, ICCPaymentProfileAdapter, ICCPaymentProfileDetailAdapter, ICCProcessingPluginFactory)`
  - `CCCustomerInformationManager.DeletePaymentProfile(PXGraph, ICCPaymentProfileAdapter, ICCPaymentProfileDetailAdapter)` with `CCCustomerInformationManager.DeletePaymentProfile(PXGraph, ICCPaymentProfileAdapter, ICCPaymentProfileDetailAdapter, ICCProcessingPluginFactory)`
  - `CCCustomerInformationManager.GetCustomerProfile(PXGraph, String, String)` with `CCCustomerInformationManager.GetCustomerProfile(PXGraph, String, String, ICCProcessingPluginFactory)`
  - `CCCustomerInformationManager.ctor(ProcessingCardsPluginFactory)` with `CCCustomerInformationManager.ctor(Object)`

- In the `PX.Objects.AR.CCPaymentProcessing.Helpers` namespace:
  - `CCPluginTypeHelper.GetProcessor<T>(CCProcessingCenter, ICardProcessingReadersProvider, CCProcessingFeature)` with `CCPluginTypeHelper.GetProcessor<T>(Object, ICardProcessingReadersProvider, CCProcessingFeature)`
  - `CCProcessingFeatureHelper.IsPaymentHostedFormSupported(CCProcessingCenter)` with `CCProcessingFeatureHelper.IsPaymentHostedFormSupported(CCProcessingCenter, ICCProcessingPluginFactory)`
  - `CCProcessingHelper.CCProcessingCenterNeedsExpDateUpdate(PXGraph, CCProcessingCenter)` with `CCProcessingHelper.CCProcessingCenterNeedsExpDateUpdate(PXGraph, CCProcessingCenter, ICCProcessingPluginFactory)`
  - `CCProcessingHelper.GetTokenizedPMsString(PXGraph)` with `CCProcessingHelper.GetTokenizedPMsString(PXGraph, ICCProcessingPluginFactory)`

  Note that the `CCProcessingFeatureHelper` class is static now.
- In the `PX.Objects.CA` namespace:
  - `PXProviderTypeSelectorAttribute.GetProviderRecs(Type[])` has been replaced with `PXProviderTypeSelectorAttribute.GetProviderRecs(Type[])`, which is now available in the `PX.Objects.Common` namespace.
  - `PXProviderTypeSelectorAttribute+ProviderRec.ctor()` has been replaced with `PXProviderTypeSelectorAttribute+ProviderRec.ctor()`, which is now available in the `PX.Objects.Common` namespace.
- In the `PX.Objects.CC.PaymentProcessing` namespace, the following methods have been replaced:
  - `Level3Processing.ctor()` with `Level3Processing.ctor(ICCProcessingPluginFactory)`
  - `Level3Processing.ctor(ICCPaymentProcessingRepository)` with `Level3Processing.ctor(ICCPaymentProcessingRepository, ICCProcessingPluginFactory)`
  - `MobileTerminalProcessing.ctor(PXGraph, IAuthorizationManager, IApplicationManager)` with `MobileTerminalProcessing.ctor(PXGraph, IAuthorizationManager, IApplicationManager, ICCProcessingPluginFactory)`
  - `PayLinkProcessing.ctor(ICCPaymentProcessingRepository)` with `PayLinkProcessing.ctor(ICCPaymentProcessingRepository, ICCProcessingPluginFactory)`

  In the `PX.Objects.Extensions.PaymentTransaction` namespace:
  - `PaymentTransactionGraph<TGraph, TPrimary>.GetClassMethodName()` with `PaymentTransactionGraph<TGraph, TPrimary>.GetClassMethodName(String)`

The following methods have been removed:

- In the `PX.Objects.AR.CCPaymentProcessing.Helpers` namespace:
  - `CCPluginTypeHelper.CreatePlugin(CCProcessingCenter)`
  - `CCProcessingFeatureHelper.ctor()`

Updating Customization Projects for the Modern UI

If your customization project introduces new Acumatica ERP forms or customized versions of existing Acumatica ERP forms built for the Classic UI, you need to convert them to the Modern UI to deliver these forms in Acumatica ERP 2026 R1 or later. You can convert these forms individually or convert all of a customization project's forms with a single action.

Updating a Customization Project for the Modern UI: General Information

This chapter describes how to convert custom and customized forms in a customization project to the Modern UI by using Acumatica ERP's conversion tools.

Learning Objectives

In this chapter, you'll learn how to:

- Convert a custom form in the Classic UI by using the form converter and include the generated Modern UI source files in the customization project
- Convert a single custom or customized form—or all the forms in the customization project at once—by using the customization project converter

Applicable Scenarios

You update a customization project for the Modern UI if you have custom or customized forms built for the Classic UI and want to convert them to the Modern UI.

Overview of Customization Files

Any form in the Modern UI—or the customization of such a form—is defined by a pair of files (TypeScript and HTML) with identical names. The file names follow this structure: `<screenID>_<customizationProjectName>` (for example, `IN202500_PhoneRepairShop`).

The TypeScript file should always contain a mixin (an extension) of the original screen class. You can generate the extension code in the *Modern UI Editor* or write it manually.

The HTML file must contain a top-level `<template>` tag. This tag can be empty or can contain the code to customize the layout. All customization tags should be located on the same level within the top-level `template` tag. You can generate an HTML extension in the *Modern UI Editor* or create it manually.

For details about creating HTML and TypeScript extensions manually, see *UI Customization Development: General Information*.

Available Built-In Converters

Acumatica ERP provides two built-in converters for converting custom and customized forms to the Modern UI:

- **Form Converter**: Converts only custom forms that are published to an instance in the Classic UI to the Modern UI. This converter is ideal for converting custom forms that don't customize any existing Acumatica ERP form.

  You start the conversion by clicking **Convert to Modern UI** on the **Customization** menu of a form. The generated Modern UI files are not automatically included in your customization project; you need to manually include them. For details, see *Updating a Customization Project for the Modern UI: Form Converter*.

> Even though all out-of-the-box Acumatica ERP forms have already been converted to the Modern UI and use it by default, the form converter is still be available for these forms if you manually switch them to the Classic UI.

- **Customization Project Converter**: Converts all custom or customized forms in a customization project to the Modern UI with a single click and automatically includes all the generated Modern UI files in the customization project.

  This converter is ideal for quickly converting all the forms in a customization project to the Modern UI.

  You can also use the converter to convert individual forms within the customization project instead of converting all the forms in the customization project at once. For details, see *Updating a Customization Project for the Modern UI: Customization Project Converter*.

> The use of these converters doesn't guarantee a perfect conversion of the forms in your customization project to the Modern UI. The converters provide **a baseline conversion that you should further fine-tune** based on your specific requirements. We strongly recommend that you:
>
> - **Make necessary backups** and use these converters **only in your development environment**.
> - **Always review the files** generated by the converters and **make any required corrections**.
> - Publish the converted forms to your production environment **only after fully testing** their functionality in your development environment.

Converting a Custom Form

To convert a custom form, you can use Acumatica ERP's form converter. It generates the required TypeScript and HTML files with most fields and view classes configured as they were in the Classic UI. Then you need to modify the template configuration in the HTML file (if necessary) and include the files in the customization project.

To convert a custom form:

1. Configure the converter. For more details, see *UI Definition in HTML and TypeScript: Form Converter*.
2. Open the custom form in Acumatica ERP.
3. On the **Customization** menu, click **Convert to Modern UI**. The system:
   a. Generates the files.
   b. Saves them in the appropriate folder in the `FrontendSources\screen\src\development\screens\` folder of the instance. The folder's name is the first two letters of the screen ID and represents the functional area.
   c. Displays a notification when the conversion has completed.

   For example, for a custom form with the *IN202510* screen ID, the files are located in the `FrontendSources\screen\src\development\screens\IN\IN202510` folder.
4. Review the TypeScript file and adjust it, if necessary. For example, you might:
   a. Remove unnecessary `import` directives.
   b. Fix any formatting issues.
   c. Adjust the name in the `viewInfo` decorator, which specifies the container name. (This name is used as an object name during the configuration of particular functionality, such as workflows and import and export scenarios.)
5. Review the HTML file and adjust it, if necessary. For example, you might:
   a. Modify the `name` attribute of the `qp-template` tag. The layout should follow the recommendations in *Form Layout: Predefined Templates*.
   b. Change the IDs so that they match the guidelines for UI component IDs. See *UI Component Guide*.

c. Remove the `wg-container` attribute if you don't have tests for the Classic UI of this form. (For details, see *Testing the Modern UI*.)

6. Include the files in the customization project: In the Customization Project Editor, click **Modern UI Files** in the navigation pane. On the *Modern UI Files* page, add the TypeScript and HTML files for the form.
7. Publish the customization project.

   During publishing, the system builds the Modern UI code and shows log messages in the Compilation window. When the customization project is published successfully, you can open the form in the Modern UI.

Converting a Customized Form

To convert a customized form, you use the customization project converter. The converter generates the Modern UI version of the original form and the Modern UI version of the customized form—and then compares them to find the difference. The converter then creates an extension of the original form in the Modern UI that includes the differences, such as added fields or modified properties of a table.

> You can use the customization project converter to convert a single customized form or all the forms in a customization project with a single click. This converter also processes any custom forms in the project. For detailed steps to perform both scenarios, see *Updating a Customization Project for the Modern UI: Customization Project Converter*.

To convert a customized form, you perform the following general steps:

1. On the *Customization Projects* (SM204505) form, open the customization project with the customized form in the Customization Project Editor.
2. On the *Screens* page, click the row for the form that you want to convert to the Modern UI and click **Convert to Modern UI** on the page toolbar.

   When the conversion has completed, the system navigates to the *Modern UI Files* page, which displays the extension files generated for the converted customized form.
3. Review the generated TypeScript and HTML extension files and make any needed corrections.

   You can edit the generated files on the *Modern UI Files* page or export the files to the `development` folder and then edit them.
4. Optional: If you've edited the generated files in the `development` folder, include these changes in your customization project.
5. Publish the customization project. During publishing, the system builds the Modern UI code and shows log messages in the Compilation window. When the customization project has been published successfully, you can open the form in the Modern UI.

Updating a Customization Project for the Modern UI: Customization Project Converter

You can use Acumatica's customization project converter to convert—with a single click—a customization project that contains custom forms or customized versions of original Acumatica forms to the Modern UI. Alternatively, you can use the converter to individually convert specific forms within the project instead of converting all the forms in the customization project at once.

Regardless of the approach you choose, the converter does the following for each customized form:

- Generates the Modern UI version of the original form and the Modern UI version of the customized form—and then compares them to find the differences

- Creates extension files for the original form in the Modern UI that include the difference, such as added fields or modified table properties
- Adds the extension files to the *Modern UI Files* files page of the Customization Project Editor

For each custom form, the converter does the following:

- Generates the Modern UI version of the custom form by converting its ASPX code to TypeScript and HTML
- Adds the generated TypeScript and HTML source files to the *Modern UI Files* files page of the Customization Project Editor

> The use of the customization project converter doesn't guarantee a perfect conversion of the forms in your customization project to the Modern UI. The converter provides **a baseline conversion that you should further fine-tune** based on your specific requirements. We strongly recommend that you:
>
> - **Make necessary backups** and use the converter **only in your development environment**.
> - **Always review the files** generated by the converter and **make any required corrections**.
> - Publish the converted forms to your production environment **only after fully testing** their functionality in your development environment.

Prerequisites for Using the Customization Project Converter

Before you use the converter, make sure the following requirements are met:

- The Modern UI has been enabled and set as the default UI in your instance. For details, see *Modern UI Development: General Information*.
- The customization project you want to convert has already been published.
- The *Customizer* role has been assigned to your user account.

  > You must have the *Customizer* role to run the converter on a customization project or on a specific form within the project.

- All Acumatica ERP features related to the form have been enabled before the conversion. This gives you the converted version that's the most similar to the original version.

  > The converter ignores any JavaScript code in ASPX files or the code in the ASPX.CS files.

Running the Customization Project Converter on a Customization Project

To run the converter on an entire customization project:

1. Open the *Customization Projects* (SM204505) form and select the row in the table for the customization project you want to convert to the Modern UI.
2. On the form toolbar, click **Convert to Modern UI**.

   The system displays a notification indicating that the conversion has started and updates it when the conversion has completed.

   As part of the conversion, the converter generates the following extension files for each customized form that it converted to the Modern UI:

   - `[SCREENID]_[CUSTOMIZATION_PROJECT_NAME]_converted.ts`, which contains the differences in TypeScript code between the original and customized versions of the form
   - `[SCREENID]_[CUSTOMIZATION_PROJECT_NAME]_converted.html`, which contains the differences in HTML code between the original and customized versions of the form

When you publish the customization project, these files are saved in the `FrontendSources\screen\src\customizationScreens\[TENANT_NAME]\screens\[FIRST_TWO_LETTERS_OF_SCREENID\[SCREENID]\extensions` folder of your instance.

The converter generates the following source files for each custom form that it converted to the Modern UI:

- `[SCREENID].ts`, which contains the initialization of views for the form
- `[SCREENID].html`, which contains the HTML layout of the form

When you publish the customization project, these files are saved in the `FrontendSources\screen\src\customizationScreens\[TENANT_NAME]\screens\[FIRST_TWO_LETTERS_OF_SCREENID\[SCREENID]` folder of your instance.

3. Open the converted customization project in the Customization Project Editor and click **Modern UI Files** in the navigation pane. The *Modern UI Files* page opens. Here you'll find a list of the TypeScript and HTML files for all the forms that were converted to the Modern UI.
4. Review the generated TypeScript and HTML files and make any needed corrections. For example, for a custom form that was converted, you may need to correct the IDs of some tags in the HTML code so that they follow established naming conventions. For details, see *UI Component Guide*.

   You can make changes to the generated files on the *Modern UI Files* page or export the files to the `development` folder (by clicking **Export to Development Folder** on the page toolbar) and edit them in an external editor. For details about using the `development` folder, see *Modern UI Development: Creating Modern UI Source Files for Custom and Customized Forms*.
5. If you use the `development` folder to modify the generated files, you need to update these files in the customization project. You do this by clicking **Detect Modified Files** on the *Modern UI Files* page.
6. Publish the customization project and open the forms in your instance to verify that they were converted correctly.

Running the Customization Project Converter on a Specific Customized Form

To run the customization converter on a specific customized form within a customization project:

1. On the *Customization Projects* (SM204505) form, open the customization project in the Customization Project Editor.
2. On the *Screens* page, click the row for the form that you want to convert to the Modern UI and click **Convert to Modern UI** on the page toolbar.

   The system displays a notification indicating that the conversion has started.

   When the conversion has completed, the system navigates to the *Modern UI Files* page, which displays the extension files that were generated for the converted form:

   - `[SCREENID]_[CUSTOMIZATION_PROJECT_NAME]_converted.ts`, which contains the difference in the TypeScript code of the original and customized versions of the form
   - `[SCREENID]_[CUSTOMIZATION_PROJECT_NAME]_converted.html`, which contains the difference in the HTML code of the original and customized versions of the form

   For example, if the customized form that you converted was the *Stock Items* (IN202500) form and the name of your customization project was *PhoneRepairShop*, the extension files generated from the conversion operation would be named as follows:

   - `IN202500_PhoneRepairShop_converted.ts`
   - `IN202500_PhoneRepairShop_converted.html`

   After you published the customization project, the converter would save these files in the `FrontendSources\screen\src\customizationScreens\[TENANT_NAME]\screens\IN\IN202500\extensions` folder of your instance.
3. Review the generated TypeScript and HTML extension files and make any needed corrections,. For example, in the HTML code, you may need to correct the ID of a `qp-fieldset` tag (so that it matches the ID

specified for this tag in the original HTML code of the form) if your customization of the form adds a UI element to an existing section of a form.

You can make changes to the generated files on the *Modern UI Files* page or export the files to the `development` folder—by clicking **Export to Development Folder** on the page toolbar—and edit the files in an external editor. For details about using the `development` folder, see *Modern UI Development: Creating Modern UI Source Files for Custom and Customized Forms*.

4. Optional: If you've edited the generated files in the `development` folder, update these files in the customization project. To do this, click **Detect Modified Files** on the *Modern UI Files* page.
5. Publish the customization project and open the form in your instance to verify that it was converted correctly.

Updating a Customization Project for the Modern UI: To Convert a Customized Form

The following activity will walk you through the conversion of a customized Acumatica ERP form from the Classic UI to the Modern UI. You'll convert the form by using the customization project converter.

> The use of the customization project converter doesn't guarantee a perfect conversion of a form to the Modern UI. The converter provides **a baseline conversion that you should further fine-tune** based on your specific requirements. We strongly recommend that you:
>
> - **Make necessary backups** and use the converter **only in your development environment**.
> - **Always review the files** generated by the converter and **make any required corrections**.
> - Publish the converted form to your production environment **only after fully testing** its functionality in your development environment.

Story

Suppose that you've customized the *Stock Items* (IN202500) form for the Smart Fix company. The customization project, developed for a previous version of Acumatica ERP, contains files created for the Classic UI. You need to convert the customized form to the Modern UI and want to use the customization project converter.

Process Overview

You will use the customization project converter to convert the customized form and generate the TypeScript and HTML extension files. You will review the generated files and make necessary changes to them. You will then publish the corresponding customization project and view the Modern UI of the customized form.

System Preparation

Before you begin converting the customized *Stock Items* (IN202500) form, do the following:

1. Complete the following prerequisite activity: *Modern UI Development: To Deploy an Instance with Custom Forms and the Modern UI*. The prepared instance contains the customized *Stock Items* (IN202500) form in the Classic UI.
2. Optional: If you plan to build code manually, perform the prerequisite actions and build the source code for the first time, as described in *Modern UI Development: To Build the Source Code of All Acumatica ERP Forms for Modern UI Development*.

Step 1: Converting the Customized Form

To convert the customized *Stock Items* (IN202500) form and generate the TypeScript and HTML extensions, do the following:

1. Publish the *PhoneRepairShop* customization project (if it's not already published).
2. On the *Customization Projects* (SM204505) form, open the *PhoneRepairShop* customization project in the Customization Project Editor.
3. On the *Screens* page, click the row for the *Stock Items* form and click **Convert to Modern UI** on the page toolbar.

   The system displays a notification indicating that the conversion operation has started.

   When the operation has completed, the system navigates to the *Modern UI Files* page, which displays the extension files that were generated for the converted form:

   - `IN202500_PhoneRepairShop_converted.ts`, which contains the difference in the TypeScript code of the original and customized versions of the form
   - `IN202500_PhoneRepairShop_converted.html`, which contains the difference in the HTML code of the original and customized versions of the form

Now you can review the generated extension files and make some necessary changes.

Step 2: Reviewing the Code of the TypeScript Extension

To review the code of the TypeScript extension that was generated in the previous step, open the `IN202500_PhoneRepairShop_converted.ts` file on the *Modern UI Files* page.

Notice the following additions in the extension file, which represents the differences between the original code of the form and its customized version:

- The `CompatibleDevices` view, which is initialized in the `IN202500_PhoneRepairShop_converted` class that extends the `IN202500` screen class
- The `UsrRepairItem` and `UsrRepairItemType` fields added in the `IN202500_ItemSettings_PhoneRepairShop_converted` view class, which extends the `ItemSettings` view class
- The `RSSVStockItemDevice` grid view class

Step 3: Reviewing the Code of the HTML Extension

To review the code of the HTML extension that was generated in the previous step, open the `IN202500_PhoneRepairShop_converted.html` file on the *Modern UI Files* page.

Notice the following additions in the extension file, which represents the differences between the original code of the form and its customized version:

- The `field` tag that adds the custom `UsrRepairItem` field after the existing `ItemType` field.
- The `field` tag that adds the custom `UsrRepairItemType` field after the custom `UsrRepairItem` field.
- The `qp-tab` tag with the *Compatible Devices* caption; this tag adds a tab with the same name after the **Price/Cost** tab on the form. This `qp-tab` tag contains the `qp-grid` tag bound to the `CompatibleDevices` view.

Due to the conversion from ASPX to HTML, the converter may sometimes add incorrect IDs for some tags or use IDs that don't follow the established naming conventions.

Update the `IN202500_PhoneRepairShop_converted.html` file on the *Modern UI Files* page as follows:

1. Make sure the values of the `after` attribute for the two `field` tags are the same as the ones shown in the following list. The ID of the fieldset to which they should be added is `fsItemDefaults-General`:
   - `<field after="#fsItemDefaults-General FIELD[name='ItemType']" name="UsrRepairItem"></field>`
   - `<field after="#fsItemDefaults-General FIELD[name='UsrRepairItem']" name="UsrRepairItemType"></field>`
2. Make sure the `qp-tab` tag is defined as follows: `<qp-tab id="tab-CompatibleDevices" after="#tab-PriceCost" caption="Compatible Devices">`.
3. Correct the ID for the `<qp-grid>` tag as follows so that the ID follows established naming conventions: `<qp-grid id="grid-CompatibleDevices" view.bind="CompatibleDevices"></qp-grid>`.

   The final code of the `IN202500_PhoneRepairShop_converted.html` file is shown below.

```
<template>
  <field
    after="#fsItemDefaults-General FIELD[name='ItemType']"
    name="UsrRepairItem">
  </field>
  <field after="#fsItemDefaults-General FIELD[name='UsrRepairItem']"
    name="UsrRepairItemType">
  </field>
  <qp-tab
    id="tab-CompatibleDevices"
    after="#tab-PriceCost"
    caption="Compatible Devices">
    <qp-grid
      id="grid-CompatibleDevices"
      view.bind="CompatibleDevices"></qp-grid>
  </qp-tab>
</template>
```

4. Save your changes to the file.

Step 4: Publishing the Project and Viewing the Form

Now you will publish the customization project and view the customized form in the Modern UI. Do the following:

1. Publish the customization project.
2. Open the *Stock Items* (IN202500) form.
3. Select the *BAT3310EX* item in the table.
4. If the form doesn't already open in the Modern UI, click **Tools > Switch to Modern UI** on the form title bar. The Modern UI for the form is displayed.
5. On the **General** tab (**Item Defaults** section), make sure the **Repair Item** check box and the **Repair Item Type** box are visible.
6. Select the **Repair Item** check box. Make sure that the **Repair Item Type** box is available because the **Repair Item** check box is selected, as shown below.

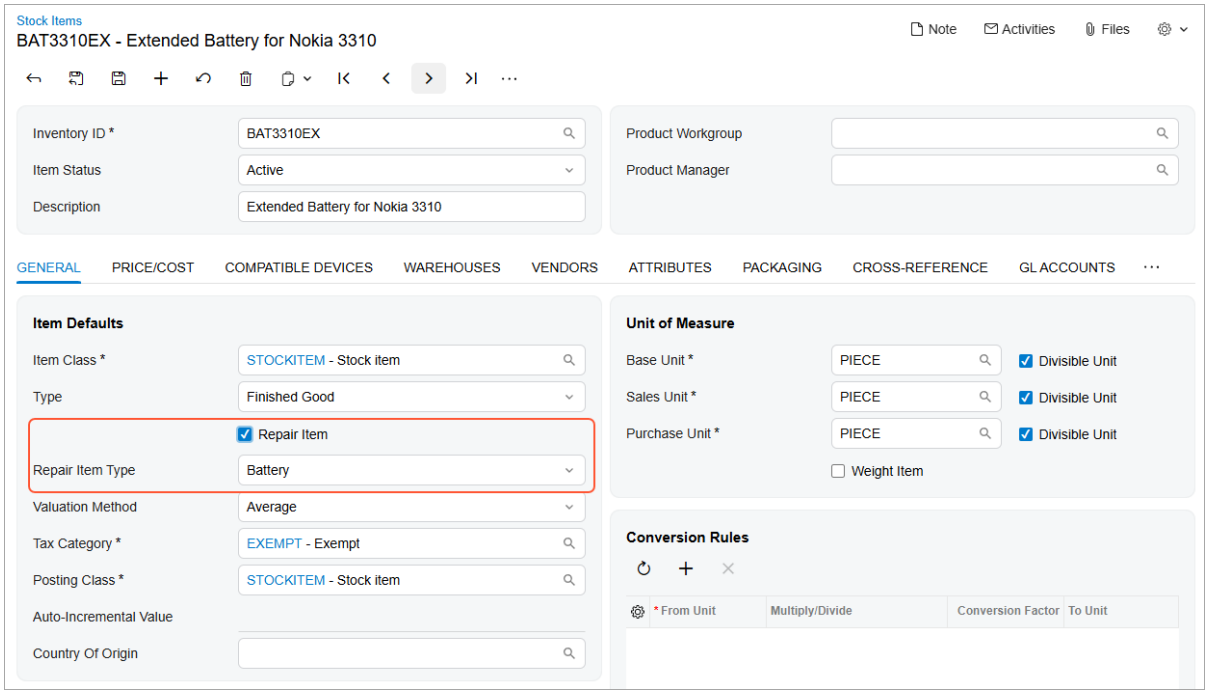

Figure: Custom boxes on the Stock Items form

7. Make sure that the **Compatible Devices** tab is displayed, as shown below. (The tab is displayed because the **Repair Item** check box is selected on the **General** tab.)

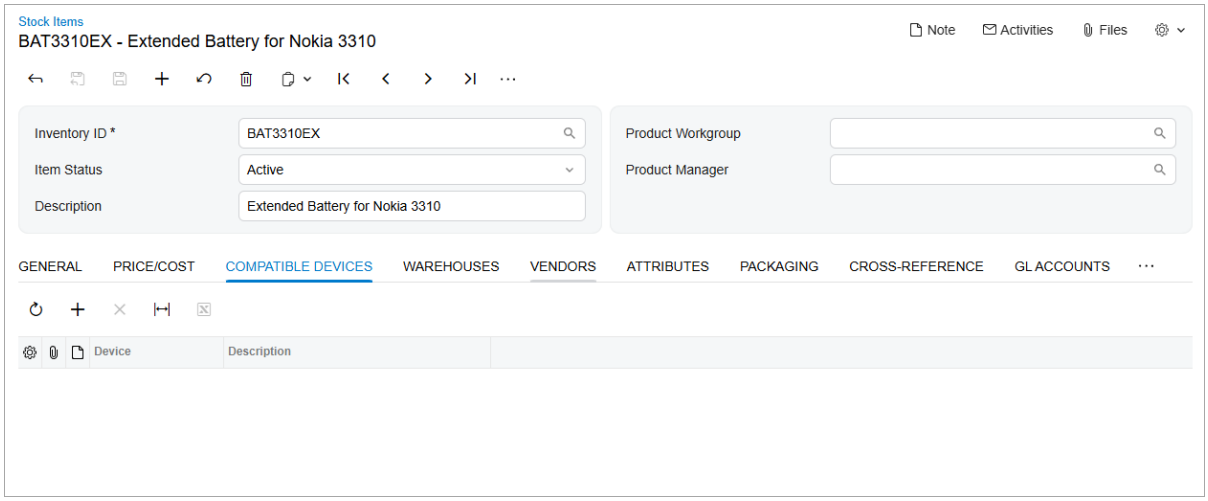

Figure: A custom tab on the Stock Items form

8. On the **Compatible Devices** tab, add the *Nokia3310* device to the list.
9. Save your changes.
10. Select the *HEADSET* item and make sure that the **Compatible Devices** tab is not displayed for this item. (This is because the **Repair Item** check box is cleared.)

Updating a Customization Project for the Modern UI: Form Converter

You can use Acumatica's form converter, which transforms an Acumatica ERP form from the Classic UI to the Modern UI. The converter parses an ASPX page of the form and generates the source files of the form in the Modern UI. These source files include a TypeScript definition and initialization of views and an HTML layout of the form.

> The use of the form converter doesn't guarantee a perfect conversion of a form to the Modern UI. The converter provides **a baseline conversion that you should further fine-tune** based on your specific requirements. We strongly recommend that you:
>
> - **Make necessary backups** and use the converter **only in your development environment**.
> - **Always review the files** generated by the converter and **make any required corrections**.
> - Publish the converted form to your production environment **only after fully testing** its functionality in your development environment.

Running the Converter

To use the converter, you need to first make sure that the Modern UI is enabled for your instance. For details on this, see *Modern UI Development: General Information*. You also need to check whether the following key exists in the `appSettings` section of the `Web.config` file of your Acumatica ERP instance: `<add key="EnableSiteMapSwitchUI" value="False" />`. If this key exists, remove it and save your changes to the file.

To run the converter, you open the needed Acumatica ERP form in the Classic UI and click **Customization > Convert to Modern UI** on the form title bar.

> The **Convert to Modern UI** command appears on the form only if the *Customizer* role is assigned to your user account.

After you execute this command, the following files are generated:

- `views.ts`, which contains declarations of all views that are used in the form
- `[SCREENID].ts`, which contains the initialization of views for the form
- `[SCREENID].html`, which contains the HTML layout of the form

By default, the files are saved in a ZIP archive, which you can download.

> - Before conversion, you need to turn on all Acumatica ERP features related to the form. This will give you the converted version that's the most similar to the original.
> - The converter ignores any JavaScript code in ASPX files or the code in the ASPX.CS files.

Configuring the Converter

You can modify the behavior of the converter by adjusting the `px.core\ui\screenConverter` tag of the `web.config` file of the instance, as shown in the following example.

```
<ui>
    <screenConverter declareViewsInViewModelFile="false" />
    </ui>
```

You can use any of the following properties of the `screenConverter` tag:

- `declareViewsInViewModelFile`: If you set this property to *True*, the converter will declare the views in the `<ScreenID>.ts` file instead of creating a separate `views.ts` file.
- `shouldFilesBeDownloaded`: If you set this property to *False*, the files are saved in the folder that's defined by the `screenConverterOutputFolder` property. By default, the `shouldFilesBeDownloaded` property is *True* and the files are saved in a ZIP archive.
- `screenConverterOutputFolder`: You can use this property to specify the output folder for the generated files. Use this property only if the `shouldFilesBeDownloaded` property is *False*. By default, the output folder is `FrontendSources\screen\src\development\screens\[FirstTwoLettersOfSCREENID]\[SCREENID]`.
- `usingOfPXJoinSyntaxEnabled`: If you set this property to *True*, the system uses the approach for joined fields, which involves the use of periods. For details on the approaches to define joined fields, see *UI Definition in HTML and TypeScript: Joined Fields*. By default, this property is *True*.
- `isAutoFormatEnabled`: If you set this property to *True*, the system uses the Prettier tool to automatically format the HTML and TypeScript code generated by the converter.

Updating a Customization Project for the Modern UI: To Convert a Custom Form

The following activity will walk you through the conversion of a custom Acumatica ERP form from the Classic UI to the Modern UI. You'll convert the form by using the form converter.

The use of the form converter doesn't guarantee a perfect conversion of a form to the Modern UI. The converter provides **a baseline conversion that you should further fine-tune** based on your specific requirements. We strongly recommend that you:

- **Make necessary backups** and use the converter **only in your development environment**.
- **Always review the files** generated by the converter and **make any required corrections**.
- Publish the converted form to your production environment **only after fully testing** its functionality in your development environment.

You can also use the customization project converter to complete this activity. For details on how to use this converter, see *Updating a Customization Project for the Modern UI: Customization Project Converter*.

Story

Suppose that you've developed the Serviced Devices (RS202000) form for the Smart Fix company. The form has been developed for a previous version of Acumatica ERP and is displayed in the Classic UI. You need to convert the form to the Modern UI and want to use the form converter.

Process Overview

You will modify the converter settings to fit your needs and convert the form to the Modern UI. You'll then review the contents of the TypeScript and HTML files and adjust them. You will include the Modern UI files in the customization project, publish it, and review the resulting form.

System Preparation

Before you begin converting the Serviced Devices (RS202000) form, do the following:

1. Complete the following prerequisite activity: *Modern UI Development: To Deploy an Instance with Custom Forms and the Modern UI*. The prepared instance contains the Serviced Devices (RS202000) form in the Classic UI.
2. Optional: If you plan to build code manually, perform the prerequisite actions and build the source code for the first time, as described in *Modern UI Development: To Build the Source Code of All Acumatica ERP Forms for Modern UI Development*.

Step 1: Adjusting the Converter Settings

The form converter comes with default settings. You need to adjust them to have the following results after conversion:

- The views are declared in the `RS202000.ts` file instead of in a separate `views.ts` file.
- The files of the Serviced Devices (RS202000) form are saved in the `FrontendSources\screen\src\development\screens\RS\RS202000` folder of the instance instead of a ZIP file.

To configure the converter, add the `shouldFilesBeDownloaded` and `declareViewsInViewModelFile` attributes in the `px.core\ui\screenConverter` tag of the `web.config` file of the instance, as shown in the following code.

```
<ui>
  <screenConverter usingOfPXJoinSyntaxEnabled="true"
    shouldFilesBeDownloaded="false"
    declareViewsInViewModelFile="true"/>
</ui>
```

> The `usingOfPXJoinSyntaxEnabled` attribute is specified in the `web.config` file by default.

Step 2: Generating the Source Files with the Converter

Now you can generate the source files of the form by using the converter. To generate the files, do the following:

1. Open the Serviced Devices (RS202000) form.
2. On the **Customization** menu, click **Convert to Modern UI**.

   The system generates the files, saves them in the `FrontendSources\screen\src\development\screens\RS\RS202000` folder of the instance, and displays a notification that the conversion has completed.
3. Close the notification by clicking **OK**.

Step 3: Adjusting the Generated TypeScript File

The generated TypeScript file may contain unnecessary `import` directives. To clean up the code, do the following:

1. Review the `RS202000.ts` file.

   The TypeScript code contains the `RS202000` screen class, which extends the `PXScreen` class and includes a property for the data view of the form. The code also contains the `RSSVDevice` view class, which extends the `PXView` class.
2. Adjust the file as follows:
   a. Remove unnecessary `import` directives.
   b. Fix any formatting issues.

c. Adjust the name in the `viewInfo` decorator, which specifies the container name. (This name is used as an object name during the configuration of the particular functionality, such as workflows and import and export scenarios.)

The resulting file looks as follows.

```
import {
 createSingle, PXScreen, graphInfo, viewInfo, PXView, PXFieldState
} from "client-controls";

@graphInfo({
 graphType: "PhoneRepairShop.RSSVDeviceMaint",
 primaryView: "ServDevices",
})
export class RS202000 extends PXScreen {
 @viewInfo({containerName: "Service Devices"})
 ServDevices = createSingle(RSSVDevice);
}

// View
export class RSSVDevice extends PXView  {
 DeviceCD : PXFieldState;
 Description : PXFieldState;
 Active : PXFieldState;
 AvgComplexityOfRepair : PXFieldState;
}
```

Step 4: Adjusting the Generated HTML File

The generated HTML file may contain unnecessary code elements and inaccurate IDs. To adjust the file, do the following:

1. Review the `RS202000.html` file.

   The HTML code includes one `qp-template` element with the `1-1` name, which organizes the elements on the form into two columns of equal width. Each column is defined with the `qp-fieldset` element, which is marked with the `slot` attribute to identify the column of the template to which the fieldset belongs. Each fieldset includes the `field` elements for the form's UI elements.

2. Adjust the file as follows:

   a. Move fields from the fieldset with `slot="B"` to the end of the fieldset with `slot="A"`, and remove the fieldset with `slot="B"`. Since the form has only four fields, it's better to organize them in one column.

   b. Change the IDs so that they match the guidelines for IDs. You can use the following IDs:

      - For the `qp-template` tag: `form-ServDevices`
      - For the `qp-fieldset` tag: `fsColumnA`

      > You can find the guidelines for IDs of particular UI components in *UI Component Guide*.

   c. Remove the `wg-container` attribute because you don't have tests for the Classic UI of the Serviced Devices (RS202000) form, which can be reused for the Modern UI. (For details about the tests, see *Testing the Modern UI*.)

   d. Fix any formatting issues.

   The resulting HTML code looks as follows.

```
<template>
```

```
  <qp-template id="form-ServDevices" name="1-1">
    <qp-fieldset id="fsColumnA" slot="A" view.bind="ServDevices">
      <field name="DeviceCD"></field>
      <field name="Description"></field>
      <field name="Active"></field>
      <field name="AvgComplexityOfRepair"></field>
    </qp-fieldset>
  </qp-template>
</template>
```

Step 5: Publishing the Project and Viewing the Form

At this point, all Modern UI files for the form are prepared. You can build the code of the form manually from the console to make sure the files are correct, or you can include the files in the customization project and publish it. When the customization project is being published, the system builds the Modern UI code and displays any errors. For details on building the code manually, see *Modern UI Development: Building the Source Code*.

To include the Modern UI source files in the customization project, publish it, and view how the converted form looks in the Modern UI, do the following:

1. Include the Modern UI files in the customization project:
   a. Open the Customization Project Editor.
   b. In the navigation pane, click **Modern UI Files**.
   c. On the *Modern UI Files* page toolbar, click **Add New Record**.
   d. In the **Add Files** dialog box, select the `RS202000.html` and `RS202000.ts` files, which are located in the `FrontendSources\screen\src\development\screens\RS\RS202000` folder.
   e. Click **Save**.
2. Publish the customization project.
3. Open the Serviced Devices (RS202000) form.
4. While you're on the Classic UI of the Serviced Devices (RS202000) form, click **Tools > Switch to Modern UI** on the form title bar. The form is shown in the Modern UI.
5. In the **Device Code** box, click the magnifier button.

   The lookup table opens, as shown below.

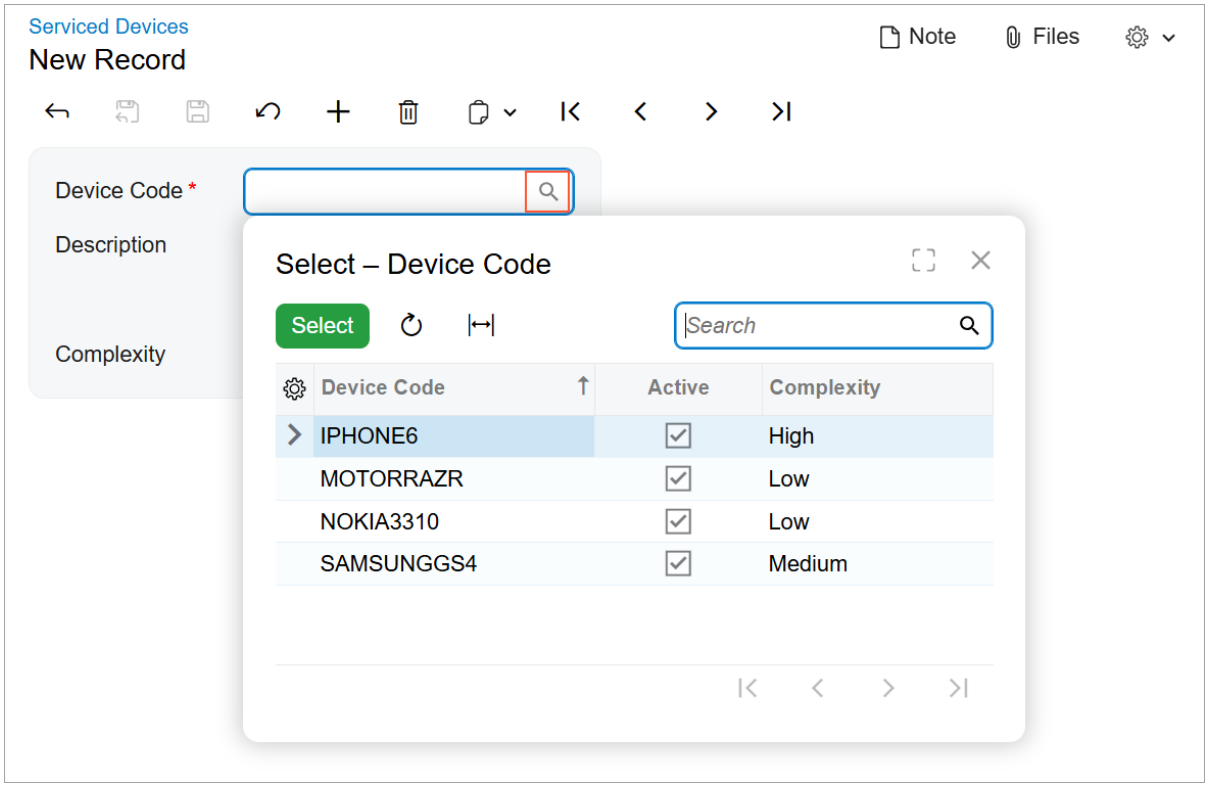

Figure: The lookup table

6. In the lookup table, select *MotorRAZR*.

   The remaining elements on the form are filled in with the *MotorRAZR* device's settings, as shown below.

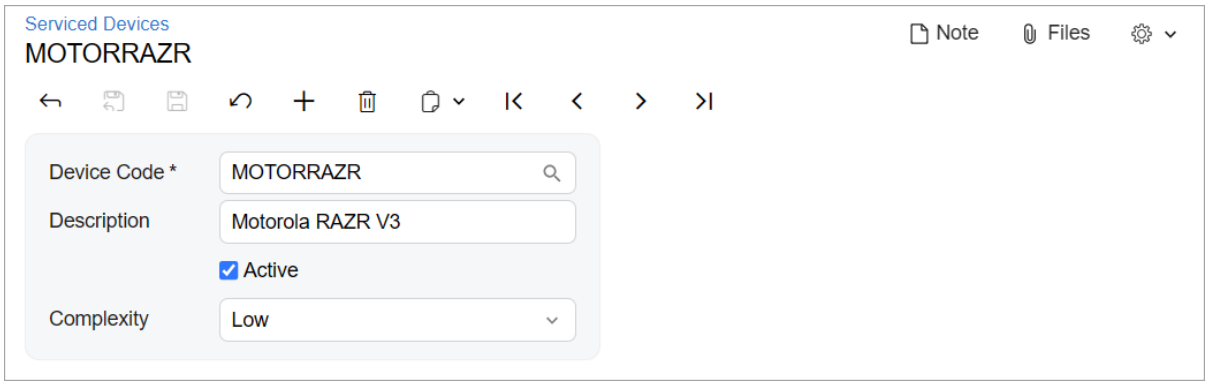

Figure: The device settings

7. Clear the **Active** check box.
8. On the form toolbar, click **Save**.